

Software Requirements Specification

for

Smart Gate Access Control System

Version 1.0

Prepared by:

Nomsa Manana Celimpilo Mdluli Sizolwethu Dlamini Nomkhosi Myeni

Client: University of Eswatini (UNESWA)

Department of Computer Science

May 12, 2026

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions and Acronyms	3
1.4	References	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	User Characteristics	5
2.3	Operating Environment	5
2.4	Design and Implementation Constraints	5
2.5	Assumptions and Dependencies	6
3	Functional Requirements	7
3.1	Gate Operations	7
3.2	Data Management	8
4	Non-Functional Requirements	9
4.1	Performance	9
4.2	Security	9
4.3	Usability	9
4.4	Reliability and Availability	9
4.5	Maintainability	10
4.6	Scalability	10
5	Interface Requirements	11
5.1	User Interfaces	11
5.2	Hardware Interfaces	11
5.3	Software Interfaces	11
5.4	Communication Interfaces	11

6	Requirements Traceability Matrix	12
7	Appendices	13
7.1	Appendix A: Database Schema Outline	13
7.2	Appendix B: Acceptance Criteria Summary	13
7.3	Appendix C: Version History	14

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the Smart Gate Access Control System. The document is intended for the development team, quality assurance personnel, university administration, and the academic supervisor. It serves as the basis for the Software Design Document (SDD) and for all subsequent development and verification activities.

1.2 Scope

The Smart Gate Access Control System is a Java-based desktop application integrated with a MySQL database. The system manages entry and exit of students, staff, and authorised personnel at university gates through barcode card scanning. It replaces manual logbooks, reduces gate passage time, and provides real-time digital audit trails.

The system includes:

- Barcode card scanning and manual ID fallback.
- Real-time activity logging (entry/exit, timestamp, gate identifier).
- An administrative dashboard with user management, reporting, and system configuration.
- Role-based access control (RBAC).

The following are excluded from the scope:

- Biometric authentication.
- Mobile application integration.
- Off-campus access points.
- Real-time integration with external HR or student records systems beyond initial data import.

1.3 Definitions and Acronyms

Term	Definition
UID	Unique identifier stored on a barcode card.
RBAC	Role-Based Access Control – restriction of system access based on user role (ADMIN, STAFF, STUDENT).
JDBC	Java Database Connectivity – standard API for Java database access.
ZXing	Zebra Crossing – open-source barcode scanning library.
Gate activity	A log record containing user ID, full name, action (ENTRY/EXIT), gate identifier, and timestamp.

Fallback mode	Manual entry of ID number when scanner or database connection is unavailable.
UAT	User Acceptance Testing.
SUS	System Usability Scale – standard questionnaire with scores 0–100.

1.4 References

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- *Smart Gate Project Plan*, University of Eswatini, Department of Computer Science, 2025.
- Oracle Corporation, *Java Code Conventions*, 2020 revision.
- ISO/IEC 25010:2021, *Systems and software quality models*.
- ZXing Project, *ZXing Core Library Documentation*, GitHub, 2023.

2 Overall Description

2.1 Product Perspective

The Smart Gate system is a standalone desktop application. It interfaces with:

- USB barcode scanners (hardware input).
- A MySQL database server (centralised storage of users, cards, and gate activities).
- University identity data (one-time import for user registration; no real-time external system integration).

2.2 User Characteristics

User Role	Technical Proficiency	Primary Tasks
Security guard	Basic computer literacy	Card scanning, manual ID entry, viewing current activity
Administrator	Moderate	User registration, report generation, dashboard monitoring, system configuration
Student / staff	Minimal	Card swipe at gate
IT maintainer	High	Database backups, server maintenance, log auditing

2.3 Operating Environment

- **Gate workstations:** Windows 10 or 11, Java Runtime Environment (JRE) 11 or later, USB port for scanner.
- **Server:** Ubuntu 22.04 LTS, MySQL 8.0.x, JDK 11 or later.
- **Development environment:** IntelliJ IDEA, Git, Apache Maven.

2.4 Design and Implementation Constraints

- The system shall be developed in Java SE (JDK 11 or later).
- The graphical user interface shall use Java Swing; no external UI frameworks are permitted.
- The database shall be MySQL 8.x with the InnoDB engine.
- All database interactions shall use JDBC with `PreparedStatement`; string concatenation for SQL is prohibited.
- Passwords shall be stored as salted SHA-256 hashes.
- The system shall operate in offline fallback mode when the database connection fails.
- Source code shall be managed using Git (GitHub repository).

2.5 Assumptions and Dependencies

Assumptions:

- The university provides stable electrical power and network connectivity at all gate points.
- Students and staff carry their barcode ID cards during campus access.
- Security guards possess basic computer training.

Dependencies:

- Availability of ZXing-compatible barcode scanners.
- The IT department provisions a MySQL server and ensures JDBC connectivity.
- The university administration supplies sample student and staff data for testing.

3 Functional Requirements

3.1 Gate Operations

FR-01 Card scanning. The system shall read a barcode or QR code from a university ID card using the ZXing library and retrieve the associated user record from the `users` table. *Acceptance criterion:* A successful scan displays the user name and ID within two seconds.

FR-02 Real-time activity logging. Upon a successful scan, the system shall insert a record into the `gate_activity` table containing user ID, full name, action (ENTRY or EXIT), gate identifier, and timestamp. The new record shall appear in the administrator dashboard within five seconds of the scan.

FR-03 Manual fallback entry. When the card scanner is unavailable or fails, the security guard shall be able to enter a user's ID number manually. The system shall log the event with a `remarks` field value "manual_entry".

FR-04 Campus statistics summary. The administrator dashboard shall display the following real-time statistics, refreshed automatically every 30 seconds:

- Total entries for the current day.
- Total exits for the current day.
- Number of users currently on campus (entries without a corresponding exit).
- Number of unique users for the current day.

FR-05 User registration. The administrator shall be able to register new users (students or staff) by providing:

- Unique ID number.
- Full name.
- Role (STUDENT, EMPLOYEE, or ADMIN).
- Email address.
- Card UID (entered manually or obtained by scanning the card).

The system shall associate the card UID with the user in the `cards` table.

FR-06 Report generation. The administrator shall generate tabular reports (displayed in a JTable and exportable to CSV) filtered by:

- Date range (daily, weekly, or custom interval).
- Gate point (Gate A, Gate B, etc.).
- User role (Student, Staff, Admin).

Report generation shall complete within 60 seconds.

FR-07 Admin dashboard navigation. The dashboard shall provide a sidebar navigation panel with links to:

- Dashboard summary (statistics).

- Gate activity log (real-time).
- User management (register, edit, deactivate).
- Reports panel.
- Settings panel (database connection parameters, institutional information).

FR-08 Authentication and session management.

- The login mechanism shall validate credentials against the `users` table using salted SHA-256 password hashing.
- A session token shall be created upon successful login and shall expire after 30 minutes of inactivity.
- Role-based access control (RBAC) shall restrict dashboard access to users with ADMIN or authorised STAFF roles.

3.2 Data Management

FR-09 Automated database backup. The system shall trigger an automated daily backup of the `smart_gate_db` database at 02:00 AM (server-side cron job). Backups shall be retained for 30 days.

FR-10 Audit log immutability. Once written, entries in the `gate_activity` table shall not be editable or deletable by any user, including administrators.

4 Non-Functional Requirements

4.1 Performance

- NFR-01 Response time.** The interval from card scan to system response (including database insert and user interface update) shall not exceed two seconds under normal operating conditions (fewer than 100 concurrent users, network latency below 50 ms).
- NFR-02 Concurrency.** The system shall support up to five gate workstations simultaneously logging activities without deadlock or data loss.
- NFR-03 Report generation time.** Any report containing up to 10,000 rows shall load within 60 seconds on the specified server hardware (8 GB RAM, 4-core CPU).

4.2 Security

- NFR-04 Password storage.** Passwords shall be stored as salted SHA-256 hashes. Plain-text passwords shall not appear in logs, network transmission, or any persistent storage.
- NFR-05 SQL injection prevention.** All database queries shall use `java.sql.PreparedStatement`. Dynamic string concatenation for SQL is forbidden.
- NFR-06 Session timeout.** Administrator sessions shall expire after 30 minutes of inactivity. Unsaved work shall be discarded.
- NFR-07 Security audit trail.** All failed login attempts and unauthorised access attempts shall be recorded in a separate `security_log` table.

4.3 Usability

- NFR-08 Training time.** Security guards with basic computer literacy shall be able to perform core operations (card scanning, manual entry, dashboard login) after no more than 15 minutes of guided instruction.
- NFR-09 Task success rate.** At least 90% of users (guards, students, and staff) shall successfully complete card scanning and manual ID entry on their first attempt.
- NFR-10 System Usability Scale (SUS).** Following User Acceptance Testing, the system shall achieve a SUS score of at least 70 out of 100.

4.4 Reliability and Availability

- NFR-11 System uptime.** The system shall maintain 99% availability during university operating hours (07:00 to 22:00) throughout the pilot deployment period.
- NFR-12 Offline fallback behaviour.** When the database connection fails, the gate client shall:
- Display a clear message: “Database offline – using fallback mode”.
 - Allow manual ID entry and cache events locally.

- Automatically synchronise cached events to the database when the connection is restored.

NFR-13 Data integrity during interruptions. No gate event shall be lost or duplicated during network interruptions. The local cache shall persist across application restarts.

4.5 Maintainability

NFR-14 Modular architecture. The source code shall follow the Model-View-Controller (MVC) pattern with clear separation:

- View: Java Swing panels (`LoginFrame`, `DashboardFrame`, etc.).
- Controller: Business logic (access control, session management, RBAC).
- Model: Data access classes (`DBConnection`, DAO classes).

NFR-15 Coding standards. All Java code shall conform to the Oracle Java Code Conventions (consistent indentation, meaningful naming, Javadoc comments on all public methods and classes).

NFR-16 Database normalisation. The MySQL schema shall be in Third Normal Form (3NF) with foreign key constraints enforced at the database level.

4.6 Scalability

NFR-17 The system shall support at least 50,000 user records and 1,000,000 gate activity rows without exceeding the performance thresholds defined in NFR-01 and NFR-03, provided appropriate indexes are applied to foreign key columns.

5 Interface Requirements

5.1 User Interfaces

- **Login screen:** Fields for username and password, a “Login” button, and visual feedback for failed authentication.
- **Gate activity panel (guard view):** A prominent scan status area, a manual ID input field, a table showing recent activity for the current day, and an indicator when fallback mode is active.
- **Administrator dashboard:** Sidebar navigation, statistics cards, a real-time activity feed, a user management table with create, read, update, and delete (CRUD) operations, and a report builder with date filters.
- All user interface components shall adhere to the defined **Theme** class (dark navy and purple colour scheme, consistent font family and sizes).

5.2 Hardware Interfaces

- Barcode scanners shall be USB Human Interface Device (HID) scanners compatible with the ZXing library (either by emulating keyboard input or via a direct API).
- Each gate workstation shall provide at least one available USB port for the scanner.

5.3 Software Interfaces

- **Database:** MySQL 8.x, accessed via JDBC Driver (Connector/J version 8.0.x or later).
- **Barcode library:** ZXing Core version 3.5.x (`zxing-core.jar`).
- **Operating system interface:** No Java Native Interface (JNI) required; scanners may be treated as keyboard input where necessary.

5.4 Communication Interfaces

- All communication between gate client workstations and the database server shall use TCP/IP over the MySQL protocol (port 3306).
- For production deployment, a VPN or local VLAN is recommended but not required for development or testing.

6 Requirements Traceability Matrix

The following table maps each functional requirement to the primary project objective as defined in the *Smart Gate Project Plan* (Section 1.3).

Requirement ID	Description	Mapped Objective
FR-01, FR-02	Card scanning with real-time logging	Reduce gate passage time to under 10 seconds
FR-03	Manual fallback entry	99% uptime with graceful fallback
FR-04, FR-06	Dashboard statistics and reports	Self-service reporting within 60 seconds
FR-05	User registration	Core functional requirement
FR-07	Dashboard navigation	Core functional requirement
FR-08	Authentication and RBAC	Security (NFR-04 to NFR-07)
FR-09, FR-10	Data backup and audit immutability	Data integrity and security
NFR-01	Response time < 2 seconds	Reduce gate passage time
NFR-08, NFR-09	Usability (training time, task success)	90% task success, 15-minute training
NFR-11	99% system uptime	Objective D (high availability)

7 Appendices

7.1 Appendix A: Database Schema Outline

Table: users

- id INT PRIMARY KEY AUTO_INCREMENT
- id_number VARCHAR(20) UNIQUE NOT NULL
- full_name VARCHAR(100) NOT NULL
- role ENUM('STUDENT','EMPLOYEE','ADMIN') NOT NULL
- password_hash VARCHAR(64) NOT NULL
- salt VARCHAR(32) NOT NULL
- email VARCHAR(100)

Table: cards

- card_uid VARCHAR(50) PRIMARY KEY
- user_id INT NOT NULL (FOREIGN KEY REFERENCES users(id))
- issued_date DATETIME DEFAULT CURRENT_TIMESTAMP

Table: gate_activity

- activity_id BIGINT PRIMARY KEY AUTO_INCREMENT
- user_id INT (FOREIGN KEY REFERENCES users(id))
- id_number VARCHAR(20) NOT NULL
- action ENUM('ENTRY','EXIT') NOT NULL
- gate VARCHAR(20) NOT NULL
- remarks TEXT
- created_at DATETIME DEFAULT CURRENT_TIMESTAMP

Table: security_log

- log_id INT PRIMARY KEY AUTO_INCREMENT
- event_type VARCHAR(50) NOT NULL
- details TEXT
- ip_address VARCHAR(45)
- timestamp DATETIME DEFAULT CURRENT_TIMESTAMP

7.2 Appendix B: Acceptance Criteria Summary

- Average gate passage time 10 seconds (measured with 50 participants).
- 100% of gate events captured with complete audit trail attributes.
- Administrator reports generated in under 60 seconds without SQL knowledge.
- 99% system uptime during operating hours (07:00–22:00).
- 90% task success rate after 15 minutes of training.

7.3 Appendix C: Version History

Version	Date	Change Description
0.1	2025-03-01	Initial draft based on project plan functional requirements.
1.0	May 12, 2026	Completed SRS – non-functional requirements, interface specifications, traceability matrix, and acceptance criteria added.